

Job Scheduling & Resource Management

Next we will delve into the world of job scheduling and resource management in high-performance computing (HPC) and other distributed environments. We'll explore how job schedulers manage tasks efficiently, prioritize them effectively, and allocate resources intelligently to optimize computing power.

What is Job Scheduling System?

A software tool that automates

- the management,
- prioritization,
- and execution

of tasks (jobs) on computing systems, especially in distributed environments like HPC clusters, data centers, and cloud setups.

Used in large-scale environments where multiple tasks and users share resources.

Ensures resources are allocated optimally.

Functions of Job Scheduling

Job Scheduling & Resource Allocation:

Scheduler determines when and where jobs run based on: Available resources (CPU cores, memory, GPUs, etc.), Job priority, Policies (e.g., fair-share, time-of-day constraints).

Ensures efficient use of resources and prevents job conflicts.

Job Execution:

Once resources are allocated, the job is dispatched for execution. Jobs may run on a single node or be distributed across multiple nodes, depending on parallelization needs.

Job Submission:

Submitted jobs are placed in a queue. Jobs wait until the required resources are available. Multiple queues may exist with varying priorities (e.g., production, interactive, low-priority jobs)

Job Queueing:

Once resources are allocated, the job is dispatched for execution. Jobs may run on a single node or be distributed across multiple nodes, depending on parallelization needs.

Functions of Job Scheduling: Post-Execution Workflow Summary

Job Monitoring:

Tracks job execution progress and resource consumption (CPU, memory, etc.).
Provides logs or status updates for users and administrators.

Job Prioritization:

Jobs are assigned priorities based on factors like: User-defined priorities, Job deadlines, Fair-share policies, Resource usage.
Scheduler determines job execution order to balance fairness and efficiency.

Job Termination:

Records output once the job finishes.
Cleans up allocated resources and notifies users of job completion or failure.
In case of failure, jobs may be retried or trigger error handling mechanisms.

Resource Fairness & Usage Policies:

Ensures fair allocation of system resources.
Prevents resource monopolization by a single user.
Fair-share scheduling – allocates resources based on past usage to ensure equitable access for all users.

Scheduling Strategies

Jobs are executed in a circular order, with each job receiving a time slice. After its time slice expires, the job is put back in the queue. This approach is often used for time-sharing systems.

Jobs are executed in the order in which they are submitted. While this is simple, it does not account for job priorities or resource utilization, which can lead to inefficient resource use.

First-Come, First-Served (FCFS):

Priority Scheduling:

Shortest Job First (SJF):

Round-Robin (RR):

Fair Share Scheduling:

Deadline-Based Scheduling:

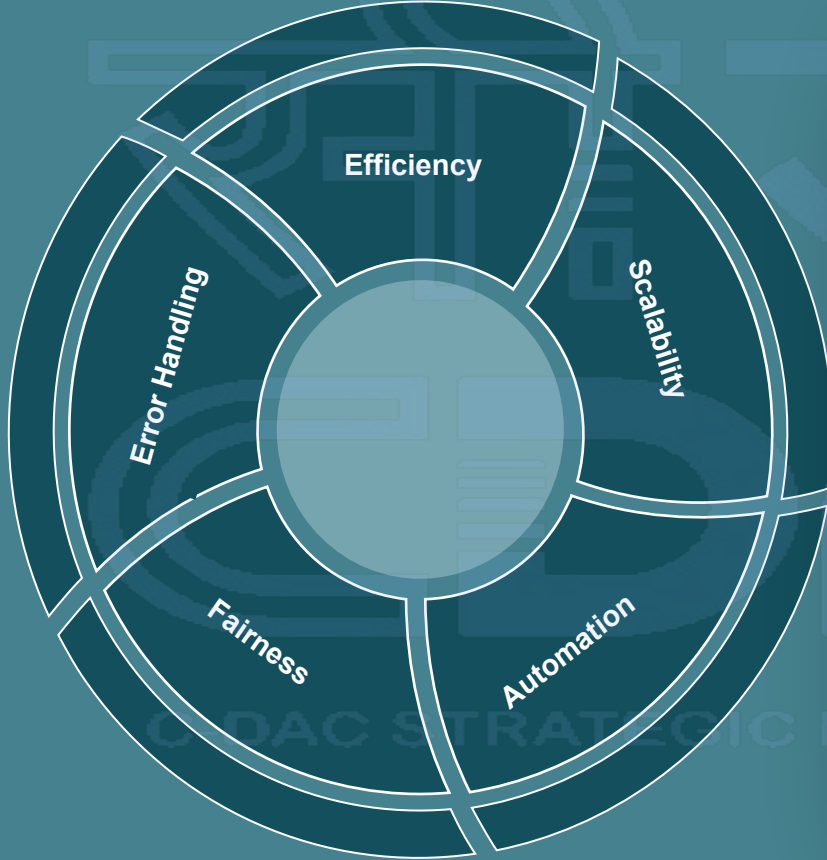
Jobs are scheduled based on a fairness policy that aims to allocate resources equitably. The system takes into account the past resource usage of users or groups to ensure no one monopolizes the system.

Jobs are assigned priority levels, and jobs with higher priority are executed first. Priorities can be determined based on user preferences, deadlines, or job importance.

Jobs are assigned deadlines, and the scheduler attempts to meet those deadlines. This is useful in environments where certain jobs must finish by a specific time, such as real-time or time-sensitive computations.

Jobs that require the least computational time are executed first. This minimizes waiting times for shorter jobs but may lead to longer job suffering from starvation.

Advantages of Job Scheduling Systems



Conclusion (Key Points)

Essential for Task Management: Job scheduling systems manage and execute tasks in distributed computing environments (HPC clusters, cloud systems, grids).

Optimize Resource Utilization: They help maximize computing resources and improve efficiency.

Improve Job Turnaround Time: Efficient scheduling ensures faster execution of tasks.

Ensure Fair Execution: They balance workloads and prioritize tasks based on predefined policies.

Widely Used Systems: Popular job schedulers include Slurm, PBS, and LSF, commonly used in research, enterprise, and cloud environments.

Who is responsible for this scheduling ?

C-DAC STRATEGIC ELECTRONICS GROUP

Resource Manager

In an HPC environment, a **Resource Manager** is a crucial component that helps in the efficient allocation and management of computational resources (such as CPU cores, memory, storage, and sometimes GPUs) across multiple users and applications. It acts as an intermediary between users or applications and the underlying hardware, ensuring resources are utilized optimally and fairly distributed.

Key Functions of Resource Manager

1

Job Scheduling

2

Resource Allocation

3

Job Prioritization

4

Monitoring

5

Queuing

6

Error Handling

7

**Resource Utilization
Optimization**

Common Resource Managers

- **Slurm (Simple Linux Utility for Resource Management)**
- PBS/Torque (Portable Batch System / Torque Resource Manager)
- LSF (Load Sharing Facility) By IBM
- Sun Grid Engine (SGE)
- Grid Engine

SLURM

Widely Used: Open-source Resource Manager & Job Scheduler for HPC clusters.

Efficient Management: Schedules jobs based on resource availability, priorities, and policies.

Job Scheduling: Slurm has a built-in scheduler with options like FIFO, fair share, and backfilling.

Advanced Resource Management:

Supports heterogeneous resources (GPUs, CPUs, memory, nodes).

Fine-grained control over job priority, partitions (queues), and user limits.

Queueing System: Handles job waiting times and prioritization.

Fairness: Implements fair-share scheduling for balanced resource access.

Scalability: Supports clusters from small setups to thousands of nodes. 🚀

Slurm Commands

C-DAC STRATEGIC ELECTRONICS GROUP

\$sinfo Partitions

```
(base) [anujas@login ~]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
standard* up 4-00:00:00      3 down* rdcn[09,11,21]
standard* up 4-00:00:00      1 drain rdcn31
standard* up 4-00:00:00     32 idle  rdcn[01-08,10,12-20,22-30,32-36]
cpu       up 8-00:00:00      3 down* rdcn[09,11,21]
cpu       up 8-00:00:00      1 drain rdcn31
cpu       up 8-00:00:00     32 idle  rdcn[01-08,10,12-20,22-30,32-36]
gpu       up 8-00:00:00      2 idle  rdgpu[01-02]
(base) [anujas@login ~]$
```

PARTITION – The name of the resource partition (queue).

- Slurm partitions group nodes with similar properties.
- Examples: standard, cpu, gpu.
- The * next to standard means it's the default partition.

AVAIL – Availability status of the partition.

- up → The partition is available for job submissions.
- down → The partition is not available.

TIME LIMIT – Maximum time limit for jobs in this partition.

- Example: 4-00:00:00 means 4 days (4d 0h 0m 0s).

NODES – Number of nodes (computers) in this partition.

STATE – Current status of nodes in the partition. Possible states include:

- idle → The node is free and available for jobs.
- down → The node is not operational (hardware failure, maintenance, etc.).
- drain → The node is being taken offline (e.g., for maintenance or admin tasks).

NODELIST – List of nodes in the partition, grouped where possible.

- Example: rdcn[09,11,21] refers to nodes rdcn09, rdcn11, and rdcn21.
- rdgpu[01-02] means GPU nodes rdgpu01 to rdgpu02.

\$squeue jobs

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
123456	standard	myjob	user1	R	00:30	2	rdcn[01-02]
123457	standard	myjob2	user2	PD	00:00	1	(Resources)
123458	cpu	analysis	user3	R	10:15	4	rdcn[05-08]
123459	gpu	deepttrain	user4	CG	02:00	1	rdgpu01

- ❖ JOBID – Unique identifier for each job.
- ❖ PARTITION – The queue (partition) in which the job is running.
 - Example: standard, cpu, gpu.
- ❖ NAME – The job name specified by the user.
 - Example: myjob, analysis, deepttrain.
- ❖ USER – The username of the job submitter.
 - Example: user1, user2.
- ❖ ST (State) – Current state of the job. Common states include:
 - R → Running
 - PD → Pending (waiting for resources)
 - CG → Completing (finalizing execution)
 - CD → Completed
 - F → Failed
 - TO → Timed Out
 - S → Suspended
- ❖ TIME – Time elapsed since the job started.
- ❖ NODES – Number of compute nodes allocated to the job.
- ❖ NODELIST(REASON) – List of nodes assigned to the job or the reason it's pending.
 - If running: Shows allocated nodes (e.g., rdcn[01-02], rdgpu01).
 - If pending: Displays reason, such as (Resources) (waiting for resources).

\$sbatch

Allocate resources through scripts

```
#!/bin/bash
#SBATCH --job-name=test_job      # Job name
#SBATCH --output=output.log      # Standard output
#                                  file
#SBATCH --error=error.log        # Error file
#SBATCH --time=01:00:00          # Time limit
#                                  (hh:mm:ss)
#SBATCH --partition=standard     # Partition (queue)
#SBATCH --nodes=1                # Number of nodes
#SBATCH --ntasks=4               # Number of tasks
#                                  (CPU cores)
#SBATCH --mem=4G                 # Memory per node

# Commands to execute
module load python                # Load Python module
python myscript.py                # Run Python script
```

Option	Description
--job-name=<name>	Sets a custom job name.
--output=<file>	Redirects standard output to a file.
--error=<file>	Redirects error messages to a file.
--time=<hh:mm:ss>	Specifies the maximum runtime.
--partition=<name>	Specifies which partition (queue) to use.
--nodes=<N>	Requests N compute nodes.
--ntasks=<N>	Number of tasks (e.g., CPU cores).
--mem=<size>	Requests memory per node (e.g., 4G, 10GB).

\$sbatch myjob.sh

\$srun

Allocate resources through interactive method

srun [options] command

Running Interactive Jobs (**srun** without **sbatch**)

You can use **srun** to request resources interactively (useful for debugging).

```
srun --partition=standard --nodes=1 --ntasks=4  
--time=00:30:00 --pty bash
```

- This requests **1 node**, **4 tasks (CPU cores)**, for **30 minutes**, and starts a new shell (**bash**).
- If a node is available, you get a shell where you can execute commands interactively.

Running a Command in a Batch Job

Inside an **sbatch** script, **srun** is used to launch parallel tasks.

Example **myjob.sh** batch script:

```
#!/bin/bash  
#SBATCH --job-name=test_job  
#SBATCH --output=output.log  
# Run a parallel task using srun  
srun --ntasks=8 gcc -fopenMP program.out
```

- This runs **myscript.py** using **8 parallel tasks** across 2 nodes.

\$scontrol

querying and modifying Slurm jobs, nodes, and partitions

```
(base) [anujas@login demo]$ scontrol show job 7536
JobId=7536 JobName=test_job
  UserId=anujas(55113) GroupId=anujas(55113) MCS_label=N/A
  Priority=80626 Nice=0 Account=cdacd QOS=normal
  JobState=RUNNING Reason=None Dependency=(null)
  Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
  RunTime=00:00:24 TimeLimit=01:00:00 TimeMin=N/A
  SubmitTime=2025-02-06T15:05:31 EligibleTime=2025-02-06T15:05:31
  AccrueTime=2025-02-06T15:05:31
  StartTime=2025-02-06T15:05:31 EndTime=2025-02-06T16:05:31 Deadline=N/A
  SuspendTime=None SecsPreSuspend=0 LastSchedEval=2025-02-06T15:05:31 Scheduler=Backfill
  Partition=standard AllocNode:Sid=login:1620541
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=rdcn[32-33]
  BatchHost=rdcn32
  NumNodes=2 NumCPUs=4 NumTasks=4 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  ReqTRES=cpu=4,mem=8G,node=2,billing=6
  AllocTRES=cpu=4,node=2,billing=6
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  MinCPUsNode=1 MinMemoryNode=4G MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
  Command=/home/anujas/demo/job_script.slurm
  WorkDir=/home/anujas/demo
  StdErr=/home/anujas/demo/error.log
  StdIn=/dev/null
  StdOut=/home/anujas/demo/output.log
  Power=
```

Summary of Common Commands

Command	Description
<code>scontrol show job <jobid></code>	View job details.
<code>scontrol cancel <jobid></code>	Cancel a job.
<code>scontrol show node <nodename></code>	View node details.
<code>scontrol update NodeName=<name> State=drain</code>	Drain a node (prevent new jobs).
<code>scontrol update NodeName=<name> State=resume</code>	Resume a drained node.
<code>scontrol show partition <partition_name></code>	View partition details.

\$scancel

Cancel a running job

```
(base) [anujas@login demo]$ squeue
      JOBID PARTITION   NAME   USER  ST       TIME  NODES NODELIST(REASON)
      7536   standard test_job  anujas  R       0:45      2 rdcn[32-33]
(base) [anujas@login demo]$ scancel 7536
bash: scancel: command not found...
Similar command is: 'scancel'
(base) [anujas@login demo]$ scancel 7536
(base) [anujas@login demo]$ squeue
      JOBID PARTITION   NAME   USER  ST       TIME  NODES NODELIST(REASON)
(base) [anujas@login demo]$
```

If you want to cancel a job that has been submitted to the queue or is currently running, you can use scancel.

Example script \$vi job_script.slurm

```
#!/bin/bash
#SBATCH --job-name=test_job      # Job name
#SBATCH --output=output.log      # Standard output file
#SBATCH --error=error.log        # Error file
#SBATCH --time=01:00:00          # Time limit (hh:mm:ss)
#SBATCH --partition=standard     # Partition (queue)
#SBATCH --nodes=2                # Number of nodes
#SBATCH --ntasks=4               # Number of tasks (CPU cores)
#SBATCH --mem=4G                 # Memory per node

cd
echo "Hello World!"

sleep 150
```

\$sacct

View Job Accounting

```
(base) [anujas@login demo]$ sacct
JobID      JobName Partition Account AllocCPUS State ExitCode
-----
7533       a.out standard cdacd    1 CANCELLED+ 0:0
7533.0     a.out standard cdacd    1 CANCELLED 0:53
7534       hello_mpi standard cdacd    4 FAILED 127:0
7534.batch batch standard cdacd    4 FAILED 127:0
7535       hello_mpi standard cdacd    4 COMPLETED 0:0
7535.batch batch standard cdacd    4 COMPLETED 0:0
7536       test_job standard cdacd    4 CANCELLED+ 0:0
7536.batch batch standard cdacd    3 CANCELLED 0:15
7537       test_job standard cdacd    4 COMPLETED 0:0
7537.batch batch standard cdacd    3 COMPLETED 0:0
(base) [anujas@login demo]$
```

The `sacct` command provides detailed information about completed jobs. You can view job status, start/end times, resources used, and more.

Summary of Common `sacct` Commands

Command	Description
<code>sacct</code>	Show all jobs for the user.
<code>sacct -j <jobid></code>	Show details of a specific job.
<code>sacct --starttime=<YYYY-MM-DD></code>	Filter jobs by start date.
<code>sacct --format=JobID,State,Elapsed</code>	Customize output fields.
<code>sacct --state=FAILED</code>	Show only failed jobs.

Submit_job.slurm

```
#!/bin/bash
#SBATCH --job-name=test_job      # Job name
#SBATCH --output=output.log      # Standard output file
#SBATCH --error=error.log        # Error file
#SBATCH --time=01:00:00          # Time limit (hh:mm:ss)
#SBATCH --partition=standard      # Partition (queue)
#SBATCH --nodes=12                # Number of nodes
#SBATCH --ntasks=40               # Number of tasks (CPU cores)
#SBATCH --mem=4G                  # Memory per node
```

```
cd
echo "Hello World!"
mpirun -np $2 $1
echo "after hello "
```

Questions?

C-DAC STRATEGIC ELECTRONICS GROUP

Question

1. What is one of the primary functions of a Resource Manager in an HPC environment?

- A) To allocate jobs to specific CPU cores
- B) To ensure optimal utilization and fair distribution of computational resources
- C) To manage storage devices for users
- D) To prioritize applications over users

C-DAC STRATEGIC ELECTRONICS GROUP

Question

2. What is indicated by the "up" status in the AVAIL field of a Slurm partition?

- A. The partition is currently unavailable for job submissions
- B. The partition has reached its maximum time limit
- C. The partition is available for job submissions and can be used immediately
- D. The partition is being drained or taken offline

Question

3. What does the NODELIST field in a Slurm partition typically group together?

- A) A list of all nodes in the partition, without any organization
- B) Nodes with similar hardware properties, such as GPUs or CPUs
- C) A list of specific nodes, grouped by their configuration (e.g., "rdgpu[01-02]")
- D) A list of users running jobs on each node

C-DAC STRATEGIC ELECTRONICS GROUP

Question

4 A scheduler determines when and where jobs run based solely on job priority.

- a. True
- b. False

Question

5. What option with sbatch command specifies which partition (queue) to use for a job?

- A) --job-name
- B) --output
- C) --time
- D) --partition

Question

6. What option with sbatch command is used to Redirects standard output to a file for a job?

- A) --job-name
- B) --output
- C) --time
- D) --partition